

Relatório de Desenvolvimento - Fase 2

Desenvolvimento de Software	INF01120
<u>Davi Santos</u>	<u>00599530</u>
<u>Gabriel Schons</u>	<u>00342020</u>
<u>Gabriel Copatti</u>	<u>00342808</u>
<u>Julio Augusto de Castilhos Borges</u>	<u>00590537</u>

Lista de Requisitos Funcionais

O sistema deve:

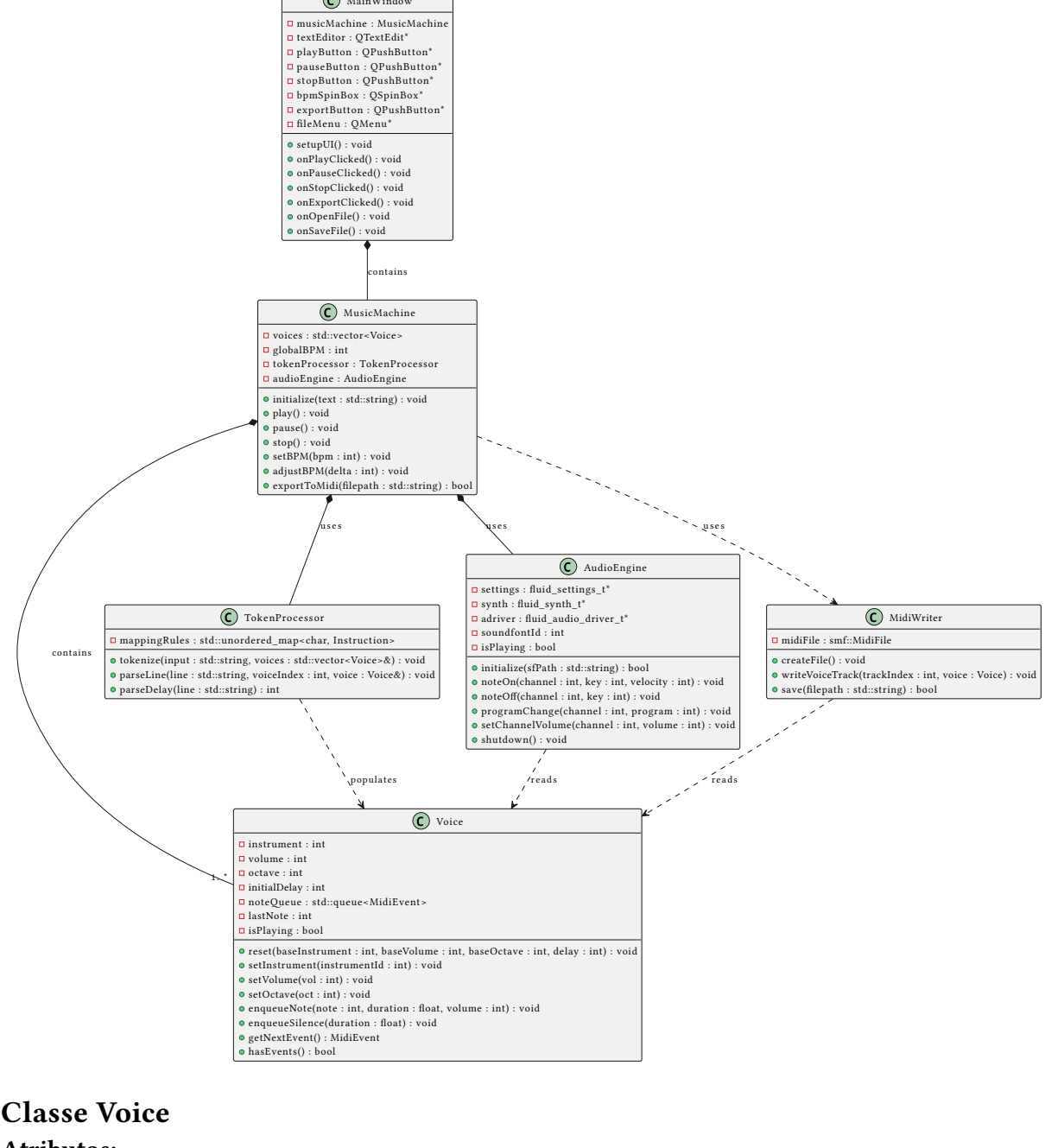
- Fornecer uma área de edição de texto na interface gráfica para que o usuário possa visualizar, editar e inserir sequências de texto livremente.
- Permitir carregar sequências de texto a partir de arquivos no formato texto simples (.txt) e salvar o texto editado de volta no mesmo formato.
- Tratar cada linha do texto de entrada como uma voz musical independente ($V_0, V_1, V_2, \dots$), processando-as de forma polifônica e simultânea.
- Mapear caracteres específicos do texto para eventos musicais locais (notas, volumes locais, instrumentos locais e alterações locais de oitava) de acordo com regras estritas de interpretação.
- Suportar atrasos iniciais individuais para cada voz através da sintaxe [n] inserida no início da linha de texto correspondente, que introduz n tempos de silêncio antes do processamento.
- Processar comandos de alteração de andamento (> e <) de escopo global, alterando o BPM do sistema para todas as vozes em tempo real durante a reprodução.
- Exportar a composição gerada para arquivos de música no formato MIDI padrão (.mid), organizando cada voz em trilhas MIDI independentes.
- Sintetizar e reproduzir o áudio resultante em tempo real utilizando a biblioteca FluidSynth com carregamento de tabelas de ondas do formato SoundFont.
- Executar a síntese de som de maneira assíncrona em uma thread dedicada separada, impedindo o bloqueio ou congelamento da interface gráfica durante o processamento de áudio.
- Permitir controlar a execução da música através de comandos de Tocar, Pausar e Parar, além de possibilitar o ajuste de parâmetros globais iniciais na interface.

Requisitos Não Funcionais

- O sistema deve seguir rigorosamente os princípios de programação orientada a objetos (SOLID).
- O código-fonte principal deve ser escrito em C++ com suporte ao padrão C++26.
- A interface gráfica deve ser construída utilizando a biblioteca Qt6 e arquivos de definição visual do Qt Designer (.ui).
- A síntese sonora em tempo real deve ser realizada através do sintetizador FluidSynth integrado ao código.
- O ciclo de vida do build da aplicação deve ser configurado via CMake, gerenciando dependências do Qt6 e FluidSynth de maneira reprodutível.
- O controle de versões do projeto deve ser gerenciado através do sistema Git.
- Testes unitários de validação da geração de eventos musicais devem ser implementados com suporte do CTest.

Definição das Classes

Diagrama UML



Classe Voice

Atributos:

- instrument: int - Armazena o ID do instrumento MIDI ativo para a trilha (0-127)
- volume: int - Armazena o volume atual da trilha (0-127)
- octave: int - Armazena a oitava local atual (0-9)
- initialDelay: int - Armazena a quantidade de batidas de atraso inicial ([n]) da voz
- noteQueue: std::queue<MidiEvent> - Filas de eventos musicais (notas e silêncios) agendados
- lastNote: int - Armazena a última nota MIDI reproduzida (para regras de repetição)
- isPlaying: bool - Indica se a voz está sendo ativamente processada

Métodos:

- reset(baseInstrument: int, baseVolume: int, baseOctave: int, delay: int): void - Restaura o estado interno e parâmetros iniciais da voz
- setInstrument(instrumentId: int): void - Modifica o instrumento da voz
- setVolume(vol: int): void - Define o volume local
- setOctave(oct: int): void - Define a oitava da voz, com limites inferior (0) e superior (9)
- enqueueNote(note: int, duration: float, volume: int): void - Adiciona um evento de nota MIDI à fila
- enqueueSilence(duration: float): void - Adiciona um evento de pausa à fila de eventos
- getNextEvent(): MidiEvent - Retorna e remove o próximo evento MIDI da fila
- hasEvents(): bool - Verifica se há eventos pendentes na fila de reprodução

Classe TokenProcessor

Atributos:

- mappingRules: std::unordered_map<char, Instruction> - Dicionário que mapeia caracteres para interpretações musicais correspondentes

Métodos:

- tokenize(input: string, voices: std::vector<Voice>&): void - Processa o texto completo de entrada, particionando em linhas e configurando a fila de notas de cada voz
- parseLine(line: string, voiceIndex: int, voice: Voice&): void - Analisa lexicalmente uma linha individual de texto, gerando e enfileirando eventos de música na voz correspondente
- parseDelay(line: string): int - Analisa o prefixo de atraso [n] no início da string, retornando o número de tempos de atraso extraído

Classe MusicMachine

Atributos:

- voices: std::vector<Voice> - Vetor contendo a lista de vozes polifônicas gerenciadas
- globalBPM: int - Armazena o andamento global em batidas por minuto (BPM)
- tokenProcessor: TokenProcessor - Referência para a classe de interpretação de texto
- audioEngine: AudioEngine - Referência para a classe gerenciadora do motor de síntese

Métodos:

- initialize(text: string): void - Prepara a máquina para reprodução realizando o parsing do texto e inicializando as vozes
- play(): void - Inicia a reprodução polifônica concorrente de todas as vozes
- pause(): void - Pausa temporariamente a execução global
- stop(): void - Finaliza a reprodução e reseta o cursor de eventos
- setBPM(bpm: int): void - Define o BPM global inicial
- adjustBPM(delta: int): void - Ajusta o BPM global dinamicamente durante a reprodução
- exportToMidi(filePath: string): bool - Exporta a sequência de eventos de todas as vozes para um arquivo padrão .mid

Classe AudioEngine

Atributos:

- settings: fluid_settings_t* - Configurações internas de alocação do sintetizador FluidSynth
- synth: fluid_synth_t* - Instância ativa do gerador de síntese FluidSynth
- adriver: fluid_audio_driver_t* - Driver de saída de áudio de tempo real do sistema operacional
- soundfontId: int - ID identificador do arquivo de SoundFont (.sf2) carregado
- isPlaying: bool - Estado atual da síntese sonora

Métodos:

- initialize(sfPath: string): bool - Configura o motor, carrega a SoundFont e ativa o driver de som
- noteOn(channel: int, key: int, velocity: int): void - Envia instrução MIDI para tocar nota no sintetizador
- noteOff(channel: int, key: int): void - Envia instrução MIDI para silenciar nota
- programChange(channel: int, program: int): void - Associa um instrumento MIDI ao canal informado
- setChannelVolume(channel: int, volume: int): void - Ajusta o volume do canal de síntese específico
- shutdown(): void - Para a síntese e desaloca recursos de memória de áudio

Classe MidiWriter

Atributos:

- midiFile: smf::MidiFile - Instância de controle de escrita de arquivos MIDI polifônicos

Métodos:

- createFile(): void - Inicializa as estruturas necessárias para o arquivo MIDI multi-track
- writeVoiceTrack(trackIndex: int, voice: Voice): void - Converte a fila de eventos de uma voz em eventos MIDI e grava em sua trilha dedicada
- save(filePath: string): bool - Salva a estrutura criada em disco no formato binário .mid

Classe MainWindow

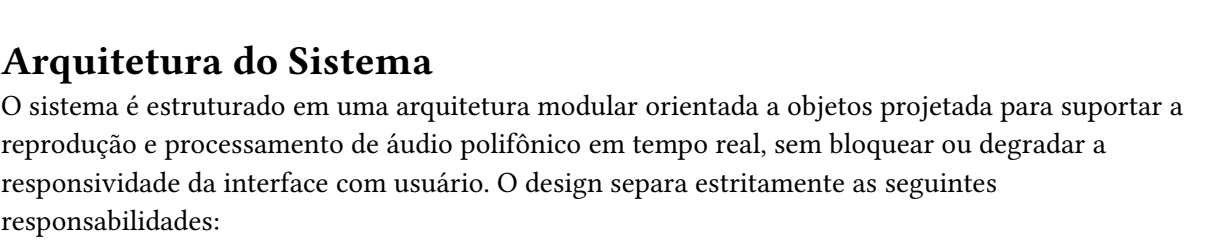
Atributos:

- musicMachine: MusicMachine - Instância da lógica central controladora da máquina
- textEditor: QTextEdit* - Componente visual da interface gráfica de edição de texto
- playButton: QPushButton* - Botão para disparar a execução
- pauseButton: QPushButton* - Botão para pausar a execução
- stopButton: QPushButton* - Botão para parar e resetar o andamento
- bpmSpinBox: QSpinBox* - Seletor numérico para configuração de BPM
- exportButton: QPushButton* - Botão para salvar arquivo de música
- fileMenu: QMenu* - Menu superior de opções de abertura/gravação de arquivos

Métodos:

- setupUI(): void - Liga e conecta dinamicamente os widgets do arquivo .ui às funções slots de controle
- onPlayClicked(): void - Inicializa a reprodução assíncrona
- onPauseClicked(): void - Pausa o thread de áudio
- onStopClicked(): void - Envia comando de interrupção e reinício de canais
- onExportClicked(): void - Solicita caminho de gravação e delega exportação
- onOpenFile(): void - Importa arquivos .txt externos para o painel de texto
- onSaveFile(): void - Salva o texto editado pelo painel para disco

GUI em Qt6



Arquitetura do Sistema

O sistema é estruturado em uma arquitetura modular orientada a objetos projetada para suportar a reprodução e processamento de áudio polifônico em tempo real, sem bloquear ou degradar a responsividade do sistema com usuário. O design separa estritamente as seguintes responsabilidades:

- Camada de Apresentação (GUI):** Representada pela classe MainWindow, consome a interface de usuário definida no arquivo qtmidi.ui gerado pelo Qt Designer. Esta camada reage a eventos e ações do usuário (carregar/salvar texto, tocar/pausar áudio, configurar parâmetros) e delega as atualizações e disparos para a camada lógica. Toda a manipulação de som e a síntese ocorrem de forma assíncrona (usando threads internas de temporização ou processamento assíncrono), prevenindo o travamento da GUI durante o tempo de execução da reprodução.
- Camada Lógica Principal:** Coordenada pela classe MusicMachine, que orquestra a leitura e execução do processador de texto, controle das vozes polifônicas e sincronismo global. A classe Voice encapsula o estado melódico independente de cada trilha de áudio (canal MIDI dedicado, oitava local, volume local, instrumento local e fila de eventos), fornecendo uma representação independente dos dados melódicos das trilhas.
- Camada de Análise Léxica e Semântica:** Implementada pela classe TokenProcessor, ela interpreta a entrada estruturada em múltiplas linhas. Ela faz o parsing de regras locais (notas A-H, pausas, textualização de oitava/volume/instrumento por caractere) de forma independente para preencher a fila de eventos de cada instância de Voice. Ela também reconhece metadados globais (como aceleração de BPM por > ou <) e inicializações especiais (como o atraso de voz [n]).
- Camada de Síntese e Driver de Áudio:** A classe AudioEngine isola a biblioteca externa FluidSynth do restante da lógica da aplicação. Ela realiza a inicialização assíncrona dos buffers, carregamento da SoundFont e despacho de eventos MIDI brutos em canais mapeados para cada voz do sistema. Isso permite que a biblioteca de áudio ou sintetizador físico seja substituído no futuro por outra tecnologia compatível sem que a lógica central da aplicação precise ser reescrita (atendendo ao princípio de Inversão de Dependência).
- Camada de Exportação:** A classe MidiWriter encapsula o empacotamento em formato de trilhas de arquivo MIDI padrão, gravando em disco o resultado estático da interpretação musical.